

Algorithms Problem Solving

Level 3

Fibonacci Series



Algorithms Problem Solving

Level 3

Fibonacci Series

Definition

Iterative Fibonacci Sequence Generation

 Program generates the first Fibonacci terms using a loop instead of recursive function calls.

 Each new value is calculated from the two preceding values.

Explanation

Fibonacci Formula

-  The sequence follows ` $F(n) = F(n-1) + F(n-2)$ `.

-  In this exercise the sequence begins with ` $1, 1$ `.

Sliding Window State

-  Two variables keep the previous two Fibonacci values.

-  After calculating the next value, the variables are shifted forward.

Iterative Control

-  A loop controls how many Fibonacci terms are generated.

-  No additional stack frames are created for every term.

Procedures

1. Initialize the first two values as `1` and `1`.
2. Print the initial values.
3. Loop until the required number of terms is generated.
4. Calculate $\text{next} = \text{prev1} + \text{prev2}$.
5. Shift the previous values forward.
6. Print the new term.

💡 Example

💡 Example

1 1 2 3 5 8 13 21 34 55

➡ Result

The first 10 Fibonacci terms are generated iteratively.

Note

- ◆ The exact starting values depend on the Fibonacci convention being used; this exercise uses `1, 1`.
- ◆ Only a small fixed number of variables is required.
- ◆ Time complexity is $O(N)$ for N generated terms.

⊘ Warning

- ◆ The order of updating `prev1` and `prev2` is important because the old values are needed to calculate the next term.
- ◆ Fibonacci numbers grow quickly and can overflow fixed-width integer types.
- ◆ The loop must account correctly for the two initial terms.



Comparison

▨ Iterative Fibonacci

- ✓ Uses $O(1)$ auxiliary memory.
- ✓ Runs in $O(N)$ time.
- ✓ Avoids recursive call-stack growth.

▤ Naive Recursive Fibonacci

- ✓ Directly follows the mathematical recursive definition.
- ✓ Recomputes many previously calculated values.
- ✓ Has exponential time complexity without memoization.

✦ Summary

- ✓ Generates Fibonacci values using a loop.
- ✓ Maintains only the two previous terms.
- ✓ Produces $O(N)$ time and $O(1)$ auxiliary space.
- ✓ Demonstrates efficient state updates in iterative algorithms.

Thank you 😊